



Universidad
Indoamérica

UNIDAD 8 · FUNDAMENTOS DE SISTEMAS OPERATIVOS

Redes: sockets

Universidad Tecnológica Indoamérica Laboratorio TD7

Dos modos de conexión

Conectado · SOCK_STREAM (TCP)

- Se establece la conexión una vez; luego se intercambian datos
- Como un **teléfono**
- Un **flujo de bytes** confiable, como una tubería: detecta errores, retransmite, conserva el orden

No conectado · SOCK_DGRAM (UDP)

- Cada envío es un mensaje independiente (**datagrama**)
- Como un **telegrama**
- Sin garantías: pérdida, duplicación, desorden

Flujo frente a datagramas, medido

Salida real

```
$ ./udp
TCP: 3 writes (one, two, three) → read #1 returns 11 bytes: "onetwothree"
UDP: 3 sendto (one, two, three) → recvfrom #1: 3 "one", recvfrom #2: 3 "two", recvfrom #3: 5
"three"
UDP: receive buffer 8192 bytes, 10000 sendto of 100 bytes succeeded, 8 received, 9992 lost
kernel counter Udp RcvbufErrors grew by 9992
```

- TCP no conserva los límites: **el protocolo de la aplicación** marca dónde termina cada mensaje (una línea, una longitud...)
- UDP pierde datagramas **sin avisar a nadie**, incluso en la misma máquina

```
int socket(int domain, int type, int protocol);
```

domain	AF_UNIX : local, la dirección es una ruta · AF_INET : IPv4 · AF_INET6 : IPv6
type	SOCK_STREAM : conectado · SOCK_DGRAM : no conectado · SOCK_RAW : paquetes crudos (requiere CAP_NET_RAW)
+ opciones	SOCK_NONBLOCK , SOCK_CLOEXEC (Linux, desde 2.6.27): se combinan con en type
protocol	0 : el sistema elige · IPPROTO_TCP · IPPROTO_UDP

Corrección (diapositiva 182): "AF_UNIX: local (127.0.0.1)" mezcla dos cosas. 127.0.0.1 es AF_INET por la interfaz de bucle local; AF_UNIX no usa IP.

Dominio local: un archivo de tipo socket

Salida real

```
$ ./local fso.sock  
bind created fso.sock: a file of type socket (the 's' of ls -l), sun_path holds 108 bytes  
the server asks the kernel who connected: pid = mine, uid 1000 (my user)  
read 5 bytes; after close the file stays until unlink  
still there after close: yes
```

- `bind` crea una entrada en el sistema de archivos (la `s` de `ls -l`); queda hasta `unlink`
- El núcleo le dice al servidor **quién** se conectó (`SO_PEERCRED`): así deciden permisos `systemd`, `D-Bus` o `/var/run/docker.sock`

Dominio Internet: dirección + puerto

- Un puerto (16 bits) identifica el servicio dentro de la máquina
- Puertos < 1024: solo con privilegios (`CAP_NET_BIND_SERVICE`); en Linux el límite es `net.ipv4.ip_unprivileged_port_start`
- Nombres de servicios: `/etc/services`

Salida real

```
$ ./addr 7000
...
ip_unprivileged_port_start = 1024
sin_port = htons(80):      bind: Permission denied
...
```

SERVICIO	PUERTO	HOY
ftp	21/tcp	sin cifrar; se usa sftp o https
telnet	23/tcp	sin cifrar; lo reemplazó ssh (22/tcp)
http	80/tcp	casi siempre redirige a https
https	443/tcp, 443/udp	TLS; HTTP/3 va sobre UDP (QUIC)

bind : dirección + puerto

C

```
int bind(int sock, const struct sockaddr *addr, socklen_t len)
struct sockaddr { sa_family_t sa_family; char sa_data[14]; }
struct sockaddr_un { /* AF_UNIX (= AF_LOCAL) */
    sa_family_t sun_family;
    char sun_path[108]; /* a path */
};
struct sockaddr_in { /* AF_INET */
    sa_family_t sin_family;
    in_port_t sin_port; /* network order */
    struct in_addr sin_addr; /* 4 bytes */
    unsigned char sin_zero[8];
};
/* AF_INET6: struct sockaddr_in6 (16-byte address) */
```

- Cada dominio tiene su estructura; todas se pasan como `struct sockaddr *` con su tamaño
- `struct sockaddr_storage` sirve para guardar cualquiera (por ejemplo, en `accept`)

Salida real

```
$ ./addr 7000
...
sizeof: sockaddr 16, sockaddr_in 16, sockaddr_in6 28,
sockaddr_un 110, sockaddr_storage 128
```

Diapositiva 187: faltaba htons

C

```
/* slide 187 */
s.sin_port = 7000;           /* wrong */
/* fixed (sock.c) */
s.sin_port = htons(7000);    /* network order */
s.sin_addr.s_addr = htonl(INADDR_ANY);
```

Salida real

```
$ ./addr 7000
port 7000 = 0x1b58; htons(7000) = 0x581b in memory order
sin_port = 7000:           the kernel bound port 22555
sin_port = htons(7000):    the kernel bound port 7000
...
```

- La red usa orden *big-endian*; x86-64 y ARM64 son *little-endian* (lo viste en el TD1 con `od`)

0x1B58 leído al revés es 0x581B = **22555**: el servidor escucha en otro puerto y nadie lo encuentra.

De `gethostbyname` a `getaddrinfo`

C tds/td7/addr.c

```
const char *names[][2] = { { "localhost", "http" }, { "
for (int k = 0; k < 2; k++) {
    struct addrinfo hints = { .ai_family = AF_UNSPEC, .
    int e = getaddrinfo(names[k][0], names[k][1], &hints,
    exit_if(e != 0, gai_strerror(e));
    for (struct addrinfo *p = res; p != NULL; p = p->ai
...
    freeaddrinfo(res);
}
```

Salida real

```
$ ./addr 7000
...
getaddrinfo("localhost", "http"): IPv4 127.0.0.1 port 80
getaddrinfo("::1", "https"): IPv6 ::1 port 443
...
```

- `gethostbyname` (diapositiva 187): solo IPv4, no es seguro con hilos, fuera de POSIX desde 2008
- `getaddrinfo` (RFC 3493): nombre + servicio
→ lista de direcciones IPv4 e IPv6, listas para `connect`

`int listen(int sock, int backlog);`

- Declara el servicio al sistema: el socket se vuelve **pasivo**, solo espera conexiones
- `backlog`: cuántas conexiones **ya establecidas** pueden esperar a `accept` (Linux admite backlog + 1)
- Con la cola llena, el SYN se descarta y el cliente lo reenvía (1 s, 2 s, 4 s...)

Salida real

```
$ ./backlog 4 8
listen(4), 8 clients connect
0.3 s, no accept yet:      accept queue 5 | clients ESTABLISHED 5, SYN_SENT 3
accept 5 (total 5), wait:  accept queue 3 | clients ESTABLISHED 8, SYN_SENT 0
accept 3 (total 8), wait: all 8 clients accepted
```

accept : tomar una conexión de la cola

```
C
int accept(int sock, struct sockaddr *a, socklen_t *len);  /* a new descriptor */
```

- `a` y `*len` se llenan con la dirección del cliente (o `NULL` si no interesa)
- Bloquea hasta que haya una conexión en la cola
- Devuelve un **descriptor nuevo** para ese cliente; el de escucha sigue aceptando

```
C tds/td7/sock.c
```

```
/* 5. accept blocks until a client connects and returns
int client = accept(s, NULL, NULL);
exit_if(client == -1, "accept");
printf("connection accepted, closing\n");
```

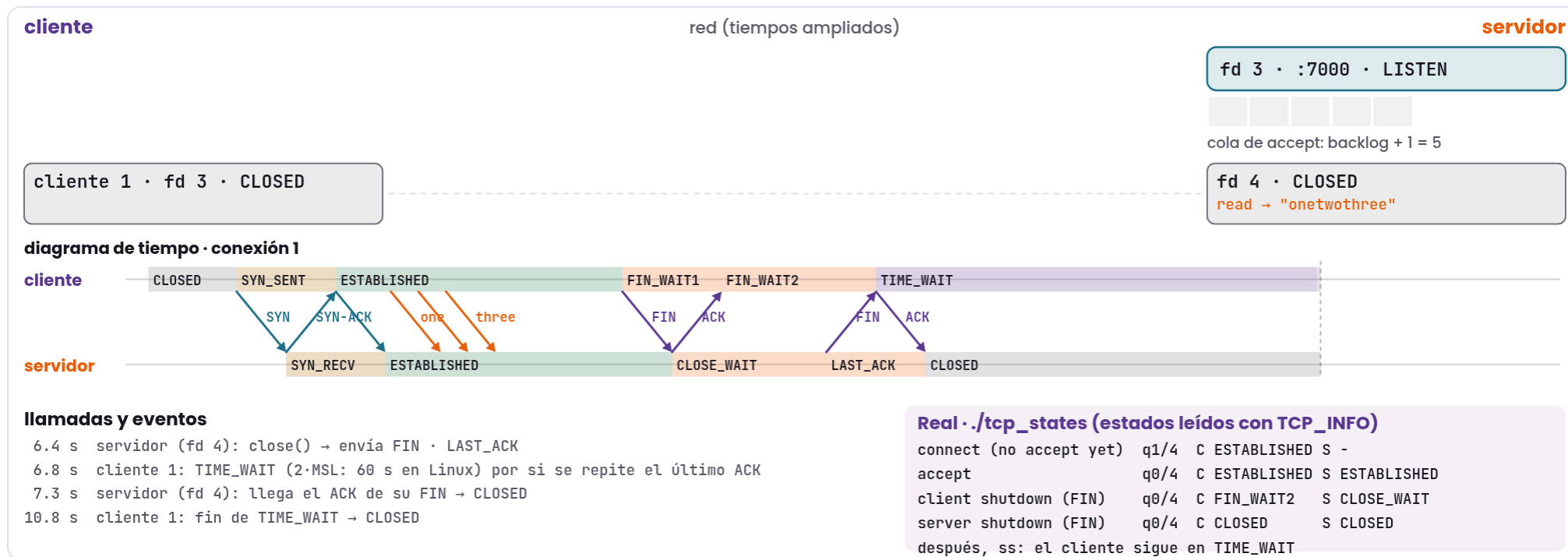
Corrección (diapositiva 189): devuelve un descriptor ≥ 0 , no > 0 (0 es válido si se cerró la entrada estándar), o -1 si hay error. `accept4(..., SOCK_CLOEXEC)` evita que el descriptor pase a un `exec`.

connect : pedir la conexión

```
C
int connect(int sock, const struct sockaddr *srv, socklen_t len);
```

- Pide la conexión a la dirección del servidor: el núcleo hace el **saludo de tres vías** (SYN, SYN-ACK, ACK)
- Bloquea hasta que el saludo termina: **no espera a** `accept`
- Errores típicos: `ECONNREFUSED` (nadie escucha en ese puerto), `ETIMEDOUT`
- Con `SOCK_NONBLOCK` devuelve `EINPROGRESS` y el saludo sigue solo (así abre `backlog.c` 8 conexiones a la vez)

Una conexión, segmento por segmento



Con el descriptor que devuelve `accept`

- Escribir: `write(fd, buf, n)` o `send(fd, buf, n, flags)`
- Leer: `read(fd, buf, n)` o `recv(fd, buf, n, flags)`
- Cerrar: `close(fd)` ; `shutdown(fd, SHUT_WR)`
cierra solo el sentido de escritura (envía FIN)

Lo que ya sabes de las unidades 2 y 5 vale aquí:

- Lecturas y escrituras **parciales**: hay que repetir (`write_all`)
- `read` → 0: el otro extremo cerró
- Escribir a un socket cerrado → `SIGPIPE` ; evítalo con `MSG_NOSIGNAL` o ignorando la señal

Los estados, leídos del núcleo con TCP_INFO

Salida real

```
$ ./tcp_states PORT; ss -tn state time-wait '( dport = :PORT )'
socket+bind+listen(4)      | listener LISTEN (queue 0/4) | client -          | server -
connect (no accept yet)    | listener LISTEN (queue 1/4) | client ESTABLISHED | server -
accept                     | listener LISTEN (queue 0/4) | client ESTABLISHED | server ESTABLISHED
write/read                  | server read 6 bytes
client shutdown (FIN)      | listener LISTEN (queue 0/4) | client FIN_WAIT2   | server CLOSE_WAIT
server read                 | server read returns 0: end of file
server shutdown (FIN)      | listener LISTEN (queue 0/4) | client CLOSED      | server CLOSED
                           | the client's end moved to a TIME_WAIT entry of the kernel (60 s, see ss)
Recv-Q Send-Q Local Address:Port Peer Address:Port
0      0      127.0.0.1:42772    127.0.0.1:PORT
```

Modo no conectado: `sendto` y `recvfrom`

C

```
ssize_t sendto(int sock, const void *buf, size_t len, int flags,  
               const struct sockaddr *dest, socklen_t dlen);  
ssize_t recvfrom(int sock, void *buf, size_t len, int flags,  
                 struct sockaddr *src, socklen_t *slen);
```

- `sendto` = `connect` + `send` en una llamada; `recvfrom` = `accept` + `recv`, y dice quién envió
- Devuelven los bytes enviados o recibidos (-1 si hay error); un `recvfrom` = **un datagrama entero** (si el búfer es corto, el resto se pierde)
- También se puede hacer `connect` sobre un socket UDP: fija el destino y permite `send` / `recv`

Quién llama a qué, en orden

CONECTADO (TCP)	
SERVIDOR	CLIENTE
socket · bind · listen	socket
accept (bloquea)	connect → saludo
read ⇔ write	write ⇔ read
close	close

NO CONECTADO (UDP)	
SERVIDOR	CLIENTE
socket · bind	socket
recvfrom (bloquea)	sendto
sendto	recvfrom
close	close

Solo el servidor hace `bind` a un puerto conocido; el cliente recibe un puerto efímero automáticamente.

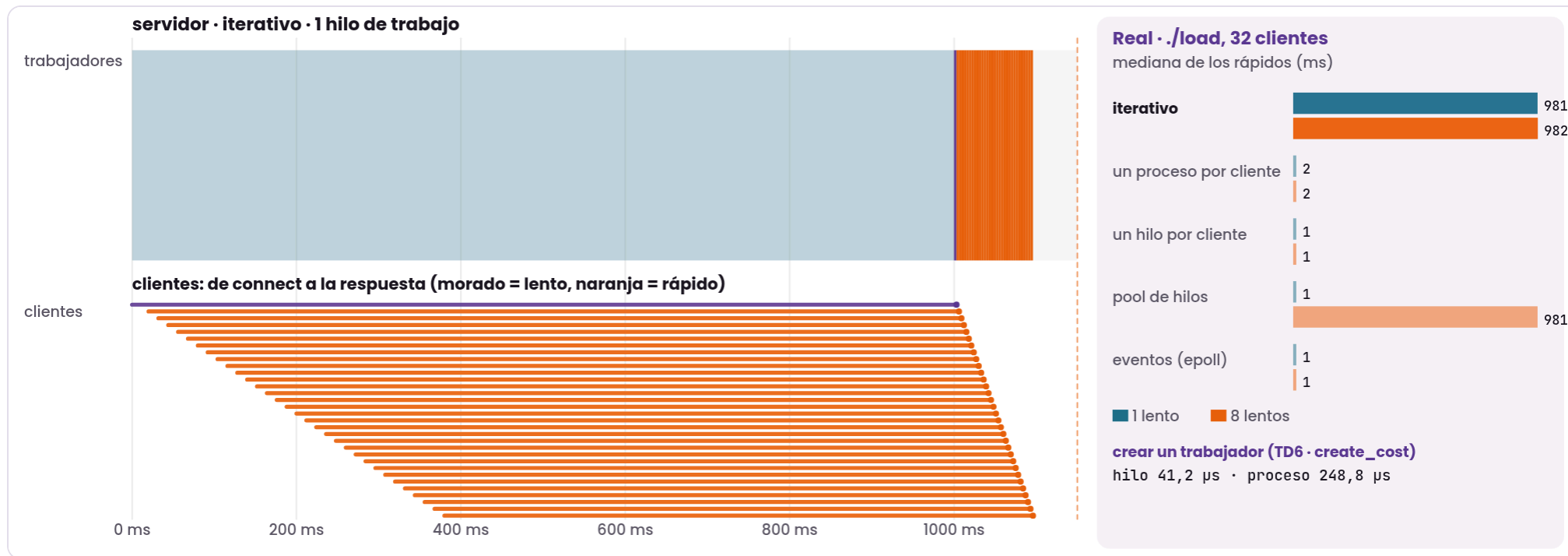
Modelos de servidor

- **Iterativo:** un cliente por vez
- **Multiproceso o multihilo:** el hilo principal hace `accept`, crea un proceso o hilo para el servicio y vuelve a `accept`
- **Pool de hilos:** igual, con un número limitado de hilos. Productor: el hilo principal (conexiones); consumidores: los hilos de servicio

Hoy además

Bucle de eventos: un hilo y `epoll` (Linux, 2002) atienden miles de conexiones; es la respuesta al problema C10K (D. Kegel, 1999) y el modelo de nginx o Node.js. `io_uring` (Linux 5.1, 2019) lleva la idea más lejos: colas compartidas con el núcleo.

¿Quién espera a quién?



32 clientes; primero 1, luego 8 lentos (1 s)

Salida real

```
$ ./http_server -m iterative & ./load PORT 1 31 1000;  
./load PORT 8 24 1000  
32 replies 200 OK of 32  
slow 1: median 1001 ms, max 1001 ms  
fast 31: median 981 ms, max 982 ms  
32 replies 200 OK of 32  
slow 8: median 1001 ms, max 1001 ms  
fast 24: median 982 ms, max 983 ms
```

Salida real

```
$ ./http_server -m epoll & ./load PORT 1 31 1000; ./load  
PORT 8 24 1000  
32 replies 200 OK of 32  
slow 1: median 1001 ms, max 1001 ms  
fast 31: median 1 ms, max 2 ms  
32 replies 200 OK of 32  
slow 8: median 1001 ms, max 1001 ms  
fast 24: median 1 ms, max 2 ms
```

El iterativo hace esperar a todos por un solo cliente lento; `epoll`, con un solo hilo, no. El pool de 8 aguanta a 1 lento pero no a 8 (A8.2).

Conclusión

- Un socket es un descriptor: `read`, `write` y `close` como con un archivo o una tubería
- TCP: flujo confiable, conexión que hace el núcleo; UDP: datagramas sin garantías
- Direcciones en orden de red (`htons`) y resueltas con `getaddrinfo`
- El modelo de servidor decide quién espera: procesos, hilos, pool o eventos

TD7 resuelto + desafío con IA

- `sock`, `echo_server` (iterativo y con `fork`)
- `http_server` con cinco modelos
- Nuevo: `tcp_states`, `backlog`, `load`, `udp`, `addr`, `local`

Bash

```
make -C tds && tds/tests/run.sh td7
```



Universidad
Indoamérica

PARA LLEVAR

Tres preguntas

1. ¿Por qué `connect` puede terminar antes de que el servidor llame a `accept`?
2. ¿Qué puerto abre `sin_port = 7000` en tu portátil, y por qué?
3. ¿Qué modelo de servidor aguanta 8 clientes lentos con un solo hilo?